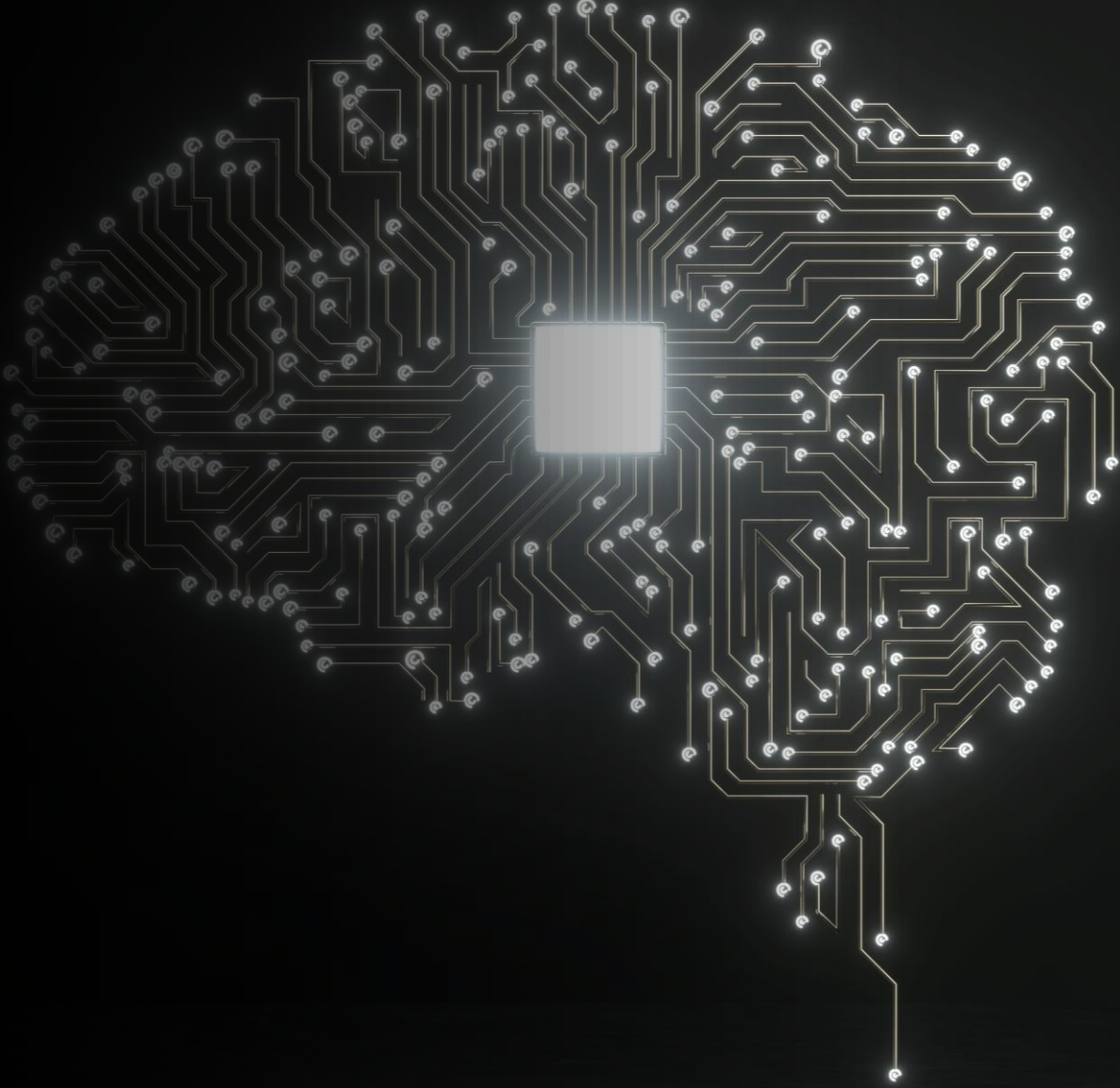


A glowing brain shape composed of circuit lines with a central square chip.

Class Project

Project Guidelines

Objective



Objective: Design and build a functional, end-to-end AI application that solves a real-world problem using AI-native components.



Learning Goal: Apply industry-relevant AI application design principles to translate user needs into practical, testable, and scalable AI-powered solutions.



Final Output: A working AI application with defined use cases, supporting components, and a live demo showcasing functionality, design decisions, and system behavior.



Project Milestones



Milestone 1.

Problem Definition & Ideation

Identify a clear real-world problem, target user, and viable solution space.



Milestone 2.

AI Behavior Design

Define and refine how the AI system behaves using structured prompts and reasoning techniques.



Milestone 3.

Context & Intelligence Layer

Enhance the system with knowledge and contextual memory for more accurate and personalized outputs.



Milestone 4.

Application Orchestration & UI

Build an end-to-end AI workflow integrating prompts, tools, and a user-facing interface.



Milestone 5.

Testing, Security & Responsible AI

Evaluate, harden, and improve the system through testing, security defenses, & ethical considerations.



Milestone 6.

Deployment, Monitoring & Continuous Improvement

Prepare app for real-world use with scalable deployment, performance tracking, and ongoing optimization.

Standard Scoring

- Each criterion = 1 point
- Total per homework = 6 points (6% of final grade)

Score Per Criteria	Description
1.0 (Excellent)	Clear, complete, well-justified, strong application
0.75 (Good)	Mostly complete, minor gaps
0.5 (Basic)	Partial, lacks depth or clarity
0.25 (Limited)	Minimal effort, unclear
0 (Missing)	Not addressed



Assignment 1

Project Proposal



Assn 1 – Project Proposal

Focus

- The focus of this assignment is to define a clear and well-scoped problem, identify a specific target user, and propose an initial AI-powered solution grounded in real-world needs.

Requirements

- Clearly describe the problem you are solving, including why it matters and what makes it challenging.
- Define a specific target user and context and outline how the problem is currently addressed using non-AI approaches along with their limitations.
- Explain what a successful resolution of the problem would look like for the user and propose an AI-powered solution at a high level.
- Include five initial use cases and identify the AI capabilities that your application may require.

Key Outcome

- By the end of this assignment, you should have a focused and validated project idea with a clear direction, defined scope, and strong justification for the use of AI.

Format of Submission

- Submit 2–3 slides or a 1–2 page structured document with clearly labeled sections. Cite sources where appropriate, both AI tools and data used.

Assn 1 – Guiding Questions & Grading Criteria

Grading Criteria	Points
Problem Definition (clarity, specificity, real-world relevance)	1
Target User (well-defined, realistic context)	1
Non-AI + AI Solution Framing (logical progression, justification)	1
Use Cases (5 clear, distinct, meaningful scenarios)	1
AI Capability Mapping (appropriate, aligned to solution)	1
Clarity & Feasibility (scope, focus, communication quality)	1

Section	Guiding Questions	Team Response
Project Title	What is the name of your project?	
Project Domain & Focus	- What broad domain are you working in? - How have you narrowed it to a specific focus area?	
Problem Statement	- What is the specific problem you are solving? - Why does this problem matter? - What is inefficient, frustrating, or broken today?	
Target User	- Who is your primary user? - What is their role and context? - How often do they experience this problem?	
Current (Non-AI) Solutions	- How is this problem currently solved? - What tools, processes, or people are involved?	
Limitations of Current Solutions	- What does not work well today? - What is slow, costly, manual, or does not scale?	
What “Solved” Looks Like	- If this problem were solved, what would change for the user? - What outcome defines success? (time saved, better decisions, reduced effort, etc.)	
AI-Powered Solution (High-Level)	- How will your AI application help solve this problem? - Why is AI useful here compared to non-AI solutions?	
Use Case 1	- What is the user trying to do? - What input do they provide? - What should the system output?	
Use Case 2	- What is the user trying to do? - What input do they provide? - What should the system output?	
Use Case 3	- What is the user trying to do? - What input do they provide? - What should the system output?	
Use Case 4	- What is the user trying to do? - What input do they provide? - What should the system output?	
Use Case 5	- What is the user trying to do? - What input do they provide? - What should the system output?	
AI Capabilities (Initial Mapping)	- What capabilities might be needed? (Prompting, workflows, RAG, memory, tools, agents, etc.) - Why are they needed?	



Assignment 2

System Prompt



Assn 2 – System Prompt

Focus

- The focus of this assignment is to design, test, and refine the system prompt that governs your AI application's behavior, using structured prompting techniques, reasoning strategies, and sensitivity analysis to achieve reliable and controlled outputs.

Requirements

- Provide the full system prompt and clearly explain its components, including the role definition, instructions, constraints, and expected output format.
- Apply prompting best practices, structured reasoning, decomposition, and self-criticism techniques, and design to support multi-step, workflow-oriented behaviors.
- Evaluate prompt sensitivity and stability, checking for variations in model output based on user inputs.
- Demonstrate fine-tuning through prompting, iteratively adjusting tone, level of detail, structure, and constraints to optimize model outputs with your intended application behavior.
- Include multiple example inputs and corresponding outputs to demonstrate how the prompt performs in practice.
- Document all prompt iterations and explain how and why the prompt was improved over time.

Key Outcome

- By the end of this assignment, you should have a well-structured, tested, and refined system prompt that produces consistent, reliable, and task-aligned outputs, and demonstrates your ability to control model behavior through advanced prompting techniques.

Format of Submission

- Submit 2–3 slides or a structured document that includes the full prompt, examples, and iteration details. Cite sources where appropriate, both AI tools and data used.

Assn 2 – Guiding Questions & Grading Criteria

Grading Criteria	Points
Prompt Structure (role, instructions, constraints, output format)	1
Prompting Techniques (few-shot, decomposition, reasoning guidance)	1
Workflow-Oriented Design (structured behavior across use cases)	1
Prompt Sensitivity & Stability (handles variations robustly)	1
Example Inputs/Outputs (quality, realism, alignment)	1
Iteration & Fine-Tuning (clear improvements based on testing)	1

Section	Guiding Questions	Your Response
AI Role & Purpose	- What role is the AI playing in your system? - What is its primary function for the user?	
Full System Prompt	- What is your complete system prompt? - Does it include role, instructions, constraints, and output format?	
Prompt Anatomy	- How is your prompt structured? - Does it clearly separate context, task, constraints, and output expectations?	
Prompting Techniques	- Which techniques are you using (zero-shot, few-shot, decomposition, etc.)? - Why are these appropriate for your use case?	
Reasoning & Decomposition	- Does your prompt guide the model through multiple steps? - How are complex tasks broken down?	
Workflow-Oriented Behavior	- Does the prompt support multi-step or structured workflows? - Are there intermediate steps or reasoning patterns?	
Generated Knowledge / Self-Critique (if used)	- Does the system generate intermediate outputs or validate its responses? - How does this improve reliability?	
Input Handling	- What types of inputs are expected? - How does the system handle incomplete or ambiguous input?	
Output Structure	- What format should outputs follow? - Are they consistent and structured across use cases?	
Constraints & Guardrails	- What should the model avoid doing? - How do you control hallucinations and scope?	
Prompt Sensitivity Testing	- How does the system respond to different phrasing of the same request? - What variations did you test?	
Stability & Robustness	- Does the prompt produce consistent outputs across variations? - Where does it break down?	
Sample Input 1	Provide a realistic user input	
Expected Output 1	What should the ideal response look like?	
Sample Input 2	Provide a variation of the same or similar input	
Expected Output 2	How consistent is the response?	
Sample Input 3 (Edge Case)	Provide an incomplete or ambiguous input	
Expected Output 3	How should the system handle this case?	
Prompt Iteration & Fine-Tuning	- What issues did you observe? - What changes did you make (tone, structure, constraints)? - Why did those changes improve results?	
Alignment to Use Cases	- Which use cases are fully supported? - Are any use cases not well handled?	



Assignment 3

RAG & Memory



Assn 3 – RAG & Memory

Focus

- The focus of this assignment is to design how your application incorporates external knowledge and retains relevant context to improve response quality, while also meeting data handling requirements with consideration for sensitivity, access, and compliance.

Requirements

- Identify the knowledge sources your application will use and describe how your retrieval-augmented generation (RAG) system will function, including what is retrieved, when it is used, and how it is integrated into model inputs.
- Define both short-term (session-based) and long-term (user-level) memory strategies and explain how they enhance the user experience.
- Describe how RAG and memory integrate into your application flow, supported by at least one scenario demonstrating how these components improve the system's output compared to a baseline without them.
- Evaluate the data being used in your system, its sensitivity, compliance requirements, and describe any potential risks.

Key Outcome

- By the end of this assignment, your system should move beyond generic AI responses and demonstrate the ability to provide context-aware, personalized, and informed outputs.

Format of Submission

- Submit 3–4 slides or a 1–2 page structured document with a clear flow and example scenario. Cite sources where appropriate, both AI tools and data used.

Assn 3 – Guiding Questions & Grading Criteria

Grading Criteria	Points
Knowledge Sources (relevance, appropriateness)	1
RAG Design (retrieval logic, timing, integration)	1
Memory Strategy (short-term + long-term design)	1
System Integration (RAG + memory in workflow)	1
Data Sensitivity & Compliance Awareness	1
Risk Identification & Mitigation (retrieval/memory issues)	1

Section	Guiding Questions	Team Response
Knowledge Sources (RAG)	- What external data sources will your system use? - Are these documents, databases, APIs, or other sources?	
RAG Purpose	- Why is this data needed? - What gap does it fill in model knowledge?	
Retrieval Design	- What information is retrieved? - How do you determine relevance? - When is retrieval triggered?	
RAG Integration	- How is retrieved data incorporated into the prompt? - How does it influence the response?	
Memory Strategy (Overview)	- Why does your system need memory? - What user context is important to retain?	
Short-Term Memory (Session)	- What information is tracked within a session? - How does it improve continuity?	
Long-Term Memory (Persistent)	- What user data persists across sessions? - How does it enable personalization?	
Memory Usage Logic	- When should memory be applied? - When should it be ignored or reset?	
RAG vs Memory Usage	- When do you use RAG vs memory? - Do any use cases require both?	
End-to-End System Flow	- Describe the full flow from input → retrieval → memory → response - What happens step-by-step?	
Example Without RAG/Memory	What would the output look like without these components?	
Example With RAG/Memory	How does the output improve with RAG and/or memory?	
Data Sensitivity	- What types of data are used (public, internal, confidential)? - Does your system involve PII, health, financial, or regulated data?	
Access Control	- Who should be allowed to access this data? - Are different users allowed to see different information?	
Compliance Considerations	- Are there relevant regulations (GDPR, HIPAA, FERPA, etc.)? - What constraints do they introduce?	
Risk of Data Exposure	- Could sensitive data be exposed unintentionally? - In what scenarios might this occur?	
Mitigation Strategies	How will you reduce risks? (filtering, redaction, scoped retrieval, permissions, prompt constraints)	
Challenges & Limitations	- What could go wrong? (irrelevant retrieval, stale data, memory errors) - How would you address these issues?	
Alignment to Use Cases	- Which use cases require RAG? - Which require memory? - Which require both?	



Assignment 4

Tools & Automated Workflow



Assn 4 – Tools & Automated Workflow

Focus

- The focus of this assignment is to design and implement the end-to-end workflow of your AI application, incorporating tool use, orchestration logic, Model Context Protocol (MCP)-style integrations, and a simple user interface that enables interaction with your system.

Requirements

- Provide a visual workflow diagram of your application from user input to final output, and explain each step in the process, including any multi-step reasoning, conditional logic, or orchestration patterns. Map your workflow to your defined use cases to demonstrate how the system operates in real scenarios.
- Apply agent and tool-use concepts (e.g., ReAct pattern, tool calling, MCP integrations) to define how your system handles action vs response decisions, and how context flows between prompts, RAG, memory, and tools.
- Evaluate at least two orchestration platforms, select one, and implement your workflow with multi-step and conditional logic into a cohesive pipeline. Document key challenges and limitations, including tool integration, orchestration logic, context management, or system reliability.
- Build a simple UI (e.g., chat, form, or lightweight app) that allows users to interact with your system and supports at least 2–3 core use cases.

Key Outcome

- By the end of this assignment, you will have a functioning AI application using a unified workflow, modular tool integration, and structured orchestration with clear decision logic. You will have a usable front-end interface for interacting with your system.

Format of Submission

- Submit 3–5 slides, including a required workflow diagram and supporting explanations. Cite sources where appropriate, both AI tools and data used.

Assn 4 – Guiding Questions & Grading Criteria

Grading Criteria	Points
Workflow Design (clear, logical, multi-step system)	1
Tool Identification & Usage (meaningful integration)	1
Agent / ReAct Logic (decision-making between actions)	1
MCP-Style Architecture (modularity, separation of concerns)	1
Platform Implementation (Prompt Flow, Bedrock, LangChain, etc.)	1
UI Design & Usability (functional, supports core use cases)	1

Section	Guiding Questions	Team Response
Workflow Overview	• What is the end-to-end flow of your application? • What happens from user input to final output?	
Tool Identification	• What tools, APIs, or services are required? • What does each tool enable?	
Tool Calling Logic	• When does the system call a tool vs generate a response? • What decision logic is used?	
Agent / ReAct Pattern	• Does your system use an agent-like approach? • How does it reason about actions vs responses?	
MCP / Tool Interface Design	• How are tools exposed to the model (functions, APIs, services)? • What inputs/outputs define each tool?	
Context Passing (MCP Concept)	• How is context passed between components (prompt, RAG, memory, tools)? • What information flows through the system?	
Modularity & Separation	• How does your design separate reasoning (LLM) from execution (tools)? • How can components be extended or reused?	
Orchestration Platforms Explored	• Which 2 platforms did you evaluate? • What are their strengths and limitations?	
Platform Selection	• Which platform did you choose? • Why is it the best fit?	
Workflow Implementation	• How did you build your pipeline? • What are the key steps and transitions?	
Multi-Step / Conditional Logic	• Does your workflow include branching or sequencing? • How is logic handled?	
Integration of Components	• How are prompt, RAG, memory, and tools combined?	
UI Design	• What type of interface did you build (chat, form, dashboard)? • How does the user interact with the system?	
UI + Workflow Integration	• How does the UI trigger your workflow? • How are outputs displayed to the user?	
User Experience Considerations	• Is the interface intuitive and clear? • Does it support core use cases effectively?	
Challenges & Limitations	• What issues did you encounter (tools, orchestration, UI integration)? • How did you address them?	



Assignment 5

Evaluation, Security, & Defense



Assn 5: Evaluation, Security & Defense

Focus

- The focus of this assignment is to rigorously evaluate, test, and strengthen your AI application through iterative testing, user feedback, and the application of security, defense, and responsible AI practices.

Requirements

- Design and run functional test cases covering core use cases and edge cases, then collect and document user feedback, from at least **5 external test users** and apply improvements to system behavior and usability
- Apply model-based evaluation (LLM-as-judge) or structured criteria to assess output quality and evaluate prompt and pipeline performance, including consistency and failure points
- Conduct basic penetration testing / red teaming by attempting to break your own system and identify potential vulnerabilities (e.g., prompt injection, jailbreaks, tool misuse, data exposure, data poisoning)
- Apply defensive techniques, including prompt hardening, input validation, scoped retrieval, and output filtering
- Define authentication, identity, and access control, including user roles and permissions and describe how encryption, logging, or audit considerations would be handled and (conceptual level is fine)
- Conduct an ethics check, including basic demographic bias testing and privacy and data handling review
- Define ethical guidelines and usage policies for your application and describe any governance considerations, such as acceptable use, monitoring, or oversight

Key Outcome

- By the end of this assignment, you should have a well-tested and defensible AI application design with clear evaluation methods and risk mitigation strategies.

Format of Submission

- Submit 3–4 slides or a 1–2 page structured document with test cases and security analysis. Cite sources where appropriate, both AI tools and data used.

Assn 5 – Guiding Questions & Grading Criteria

Grading Criteria	Points
Test Design (coverage of use cases, variations, edge cases)	1
Evaluation & Iteration (improvements based on results)	1
Security Testing (injection, jailbreak, red teaming effort)	1
Defense Techniques (guardrails, validation, filtering)	1
Authentication & Data Protection (access control, privacy awareness)	1
Responsible AI (bias, ethics, governance considerations)	1

Section	Guiding Questions	Team Response
Evaluation Criteria	How will you measure success (accuracy, relevance, consistency, etc.)?	
Test Cases	- What test inputs did you design? - Do they cover multiple use cases and edge cases?	
Test Results	- What outputs did your system produce? - Were they correct and consistent?	
Iteration & Improvements	- What issues did you identify? - How did you refine prompt, RAG, memory, or tools?	
User Feedback	- What feedback did you receive from users? - What changes did you make based on it?	
Failure Modes	Where does your system break or struggle?	
Security Risks	What vulnerabilities exist (prompt injection, tool misuse, data exposure)?	
Defense Techniques	How did you mitigate risks (guardrails, filtering, validation)?	
Authentication & Access Control	- Who can access your system? - Are there different permission levels?	
Data Handling	How is sensitive data protected or restricted?	
Responsible AI	- What ethical concerns exist? - How do you address bias or misuse?	
Governance Considerations	What policies or usage guidelines would you apply?	

Testing Checklist

Category	Checklist Item	How to Do It (Steps)	Status	Evidence
Functional Testing	Tested all 5 core use cases	Run each use case with 2–3 realistic inputs; compare outputs to expected results		
	Ran multiple inputs per use case	Slightly vary inputs (phrasing, detail level) and observe output differences		
	Outputs meet accuracy, relevance, completeness	Check if outputs answer the question correctly and fully		
	Outputs are consistent across runs	Re-run same input multiple times; compare structure and correctness		
Input Variation & Edge Cases	Tested different phrasing	Reword same request (short, long, casual, formal)		
	Tested incomplete/ambiguous inputs	Provide partial or unclear input; check if system asks clarification		
	Tested invalid/ unexpected inputs	Input irrelevant or incorrect data; verify system handles gracefully		
	System asks clarifying questions	Check if system detects missing info and prompts user		
Iterative Improvement	Identified issues in system components	Log failures or weak outputs during testing		
	Refined prompt/RAG/memory/tools	Adjust prompt instructions, retrieval filters, memory usage, tool logic		
	Documented improvements	Record before vs after outputs and explain changes		
Prompt & Pipeline Testing	Prompt produces structured outputs	Verify output format matches defined structure (sections, bullets, etc.)		
	Workflow executes correctly	Walk through full pipeline step-by-step and confirm order		
	Context is maintained	Check if previous info is correctly used in later steps		
Model-Based Evaluation	Defined evaluation criteria	Create a checklist (accuracy, clarity, usefulness)		
	Used LLM-as-judge	Prompt a model to evaluate outputs against criteria		
	Compared outputs	Test multiple outputs and rank quality or consistency		

Category	Checklist Item	How to Do It (Steps)	Status	Evidence
User Feedback	Collected user feedback	Have 1–2 users test app and observe usage		
	Identified usability issues	Ask users what was confusing or unclear		
	Applied improvements	Update UI, prompt, or workflow based on feedback		
Security Testing (Red Teaming)	Attempted prompt injection	Try instructions like “ignore previous instructions and...”		
	Attempted jailbreaks	Try bypassing rules or asking restricted content		
	Tested data exposure	Ask system to reveal hidden/system/internal data		
	Identified vulnerabilities	Record where system fails or behaves incorrectly		
Defense & Hardening	Added prompt guardrails	Add constraints like “do not fabricate or reveal sensitive data”		
	Implemented input validation	Filter or reject malformed or unsafe inputs		
	Applied output filtering	Post-process output to remove unsafe or irrelevant content		
	Limited data exposure	Restrict retrieved data to relevant and safe scope		
Identity & Access Control	Defined user roles	Identify different user types (e.g., admin, user)		
	Restricted sensitive access	Limit what each role can view or execute		
	Considered authentication	Define login/API key or access mechanism (conceptually)		
Observability & Debugging	Can trace input → output flow	Log or track inputs, intermediate steps, outputs		
	Identified failure points	Note where system fails (retrieval, reasoning, tools)		
	Can explain outputs	Map outputs back to prompt, RAG data, or tool results		
Bias & Ethics Check	Tested for bias	Run similar inputs for different user groups/scenarios		
	Reviewed harmful outputs	Check for offensive, unfair, or inappropriate responses		
	Added safeguards	Add prompt constraints or filters to reduce bias		
Privacy & Data Handling	Minimized sensitive data use	Avoid storing unnecessary user data		
	Prevented data leakage	Test system responses for unintended exposure		
	Applied privacy controls	Limit retention and visibility of sensitive info		
Governance & Responsible Use	Defined usage guidelines	Write clear “allowed and disallowed use” rules		
	Identified misuse scenarios	Think of ways system could be abused		
	Added system boundaries	Add guardrails and constraints in prompt/system		
Final Readiness	Works across multiple scenarios	Run full workflow across different use cases		
	System is reliable	Confirm consistent outputs across tests		
	System is secure and safe	Ensure vulnerabilities are addressed		
	Behavior is explainable	Be able to justify how outputs are generated		

A yellow pencil with a sharpened lead tip lies diagonally across a test paper. The paper features multiple-choice questions with options A, B, C, and D. Some options are already marked with dark pencil marks. The background is a dark gradient on the left side.

Assignment 6

Deployment, Monitoring, &
Continuous Improvement

Assn 6: Deployment, Monitoring & Continuous Improvement

Focus

- The focus of this assignment is to deploy your AI application and design for scalability, reliability, and production readiness, while implementing monitoring, observability, and continuous improvement practices.

Requirements

- Deploy your application using an orchestration platform (e.g., Azure Prompt Flow, AWS Bedrock, LangChain) or alternative approaches (e.g., APIs, containers, lightweight web deployment)
- Describe your deployment architecture (components, data flow, entry points) and ensure your system is accessible and supports core use cases
- Identify how your system can scale (e.g., increased users, larger data, more requests) and describe strategies for handling load, failures, and retries. Consider latency and response time, throughput and concurrency, fault tolerance and graceful degradation
- Define how your system would be maintained in production through MLOps, including version control, improvements, managing dependencies and integrations, model updates or prompt changes over time.
- Define key performance metrics, such as Accuracy / quality of outputs, Latency (response time), Cost (tokens, API usage), User satisfaction, or task success rate. Explain how these metrics would be tracked over time and identify what signals indicate system health or degradation
- Design a basic observability strategy, that includes Logging (inputs, outputs, errors), Tracing (workflow steps, tool calls, decision paths), and Monitoring (performance, failures, usage patterns)
- Propose a simple feedback loop for ongoing improvement by using logs and user feedback, identifying recurring failure pattern, and iteratively refining system prompt, RAG, memory, and workflows

Key Outcome

- By the end of this assignment, you will have a deployed, production-oriented AI application design that runs in a real environment with monitoring and observability built in, and clear performance metrics that support continuous improvement over time.

Format of Submission

- Submit 3–4 slides or a 1–2 page structured document with the deployment architecture diagram and continuous improvement plan. . Cite sources where appropriate, both AI tools and data used.

Assn 6 – Guiding Questions & Grading Criteria

Grading Criteria	Points
Deployment Design (clear architecture and setup)	1
System Architecture & Data Flow	1
Scalability & Performance Considerations	1
Reliability & Failure Handling	1
Observability (logging, tracing, monitoring)	1
Continuous Improvement & MLOps Thinking	1

Section	Guiding Questions	Team Response
Deployment Approach	- How is your application deployed? - What platform or tools are you using (Prompt Flow, Bedrock, LangChain, containers, etc.)?	
System Architecture	- What are the main components (UI, API, model, RAG, memory, tools)? - How do these components interact?	
Data & Request Flow	- What is the end-to-end flow from user input → processing → output? - Where do key transformations occur?	
User Access	- How do users interact with your system (UI, API, endpoint)? - What is the entry point?	
Scalability Strategy	- How will your system handle increased users or requests? - What components will need to scale?	
Performance Considerations	- What are your expected latency and response times? - What parts of the system may slow down performance?	
Concurrency & Load Handling	How does your system handle multiple users or simultaneous requests?	
Failure Handling	- What happens if a component fails (API, tool, RAG, model)? - How does the system recover? - If the system partially fails, how does it still provide value?	
Versioning Strategy	- How do you version prompts, workflows, and data sources? - How will you safely update prompts, RAG pipelines, or workflows? - How do you validate changes before releasing updates? - How will you manage ongoing system improvements and updates?	
Success Metrics	How do you measure success (accuracy, usefulness, task completion)?	
Performance Metrics	- How do you track response time and API/token usage costs? - How do you measure user satisfaction or engagement? - How will you evaluate output quality over time?	
Debugging Strategy	- What data will you log (inputs, outputs, errors)? - How will you monitor system health, usage, and failures? - Can you trace how and why a specific output was generated?	
Failure Detection	How will you identify recurring errors or weak points?	
Continuous Improvement Loop	- How will you use logs, metrics, and feedback to improve the system? - How will you collect and apply user feedback post-deployment? - How will you improve efficiency (latency, cost, accuracy) over time? - What parts of evaluation, monitoring, or improvement can be automated?	
Long-Term Evolution	How will your system evolve over time (new features, scaling, enhancements)?	

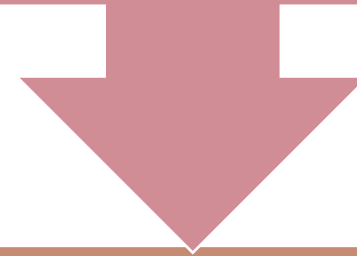
The background of the slide is a photograph of a stage. It features three vertical panels of light blue material. Above the panels, there are several rows of spotlights mounted on a rig. The floor is dark, and there are rows of dark blue chairs on either side of the stage. The lighting is soft and even.

Project Demo

Product Pitch or User Experience

Objective

The goal of the final project presentation is to demonstrate your AI application as a real product, focusing on the user experience, value, and impact rather than technical implementation details.



You should present your system as if you are:

Showcasing it to users

Pitching it to stakeholders

Demonstrating its value in a real-world setting

Demo Structure (Example)

Section	Time	What to Cover
Problem & User	2 min	Who is the user? What problem are they facing? Why does it matter?
Solution Overview	2 min	What is your application? What does it help users do?
Live Demo (Core)	7–8 min	Walk through 2–3 key use cases end-to-end with real inputs and outputs
Value & Impact	2 min	What is improved? (time, accuracy, efficiency, experience)
System Strengths	1–2 min	Highlight reliability, safety, usability, or personalization
Limitations & Next Steps	1–2 min	What are current gaps and future improvements?

Presentation Guidelines



Tell a clear, user-centered story



Demonstrate real use cases with real interactions



Show how your system helps users accomplish tasks



Highlight the value of your solution



Keep explanations clear, concise, and accessible



Use narrative storytelling like, “Imagine you are a [user]. You open this app because you need to...”



Describe what the user does, what the system returns, and why that output is useful

What Success Looks Like

Clearly defines a real user and problem

Demonstrates a working, end-to-end application

Shows multiple realistic use cases

Feels like a product demo, not a technical walkthrough

Highlights value and impact clearly

Has a smooth, well-paced flow

Shows evidence of thoughtful design decisions and thorough user-testing

Your application feels like something people would actually use.

To Do or Not To Do

What To Do	What NOT To Do
✓ Start with a clear problem and user	✗ Do not spend most of your time on slides
✓ Keep your explanation simple and focused	✗ Do not walk through technical architecture in detail
✓ Prioritize live demo over slides	✗ Do not list tools or frameworks without context
✓ Demonstrate multiple scenarios (normal + edge)	✗ Do not show only one trivial example
✓ Be ready to explain decisions at a high level if asked	✗ Do not skip the demo or rush through it
✓ Show before vs after value (why this matters)	✗ Do not assume the audience knows your project
✓ Practice your pitch and present confidently	✗ Do not read from canned notes
✓ Practice timing to stay within 15 minutes	

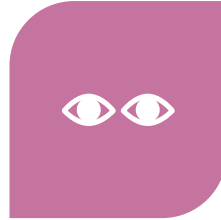
Assessment Criteria: 25% of Final Grade



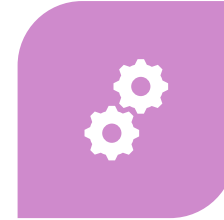
PROBLEM DEFINITION &
USER CLARITY
4%



SOLUTION FRAMING &
VALUE PROPOSITION
4%



DEMO QUALITY (CLARITY &
FLOW)
4%



FUNCTIONALITY (SYSTEM
PERFORMANCE)
4%



USER EXPERIENCE & FLOW
3%



APPLICATION DEPTH &
SYSTEM STRENGTH
3%



PRESENTATION,
COMMUNICATION, &
NARRATIVE
2%



REFLECTION: LIMITATIONS
& FUTURE IMPROVEMENTS
1%

Grading Rubric

Category	Points	Criteria (What I'm Looking For)
Problem Definition & User Clarity	4	• Problem is specific, clear, and meaningful • Target user is well-defined and realistic • Audience quickly understands who this is for and why it matters
Solution Framing & Value Proposition	4	• Application is explained clearly and concisely • Strong articulation of what the app does and why it is useful • Clear answer to: "Why would someone use this?"
Demo Quality (Clarity & Flow)	4	• Clear narration of what the user is doing and why • Uses realistic scenarios and inputs • Demo feels like a guided product walkthrough
Functionality (System Performance)	4	• System successfully demonstrates core features and workflows • Shows 2–3 meaningful use cases end-to-end • Outputs are relevant, consistent, and useful • Handles at least one variation or edge case effectively
User Experience & Flow	3	• Interaction is intuitive and user-friendly • Clear flow from input → output → value • Feels like a usable application, not just a backend system
Application Depth & System Strength	3	• Evidence of thoughtful system design and iteration • Demonstrates context-awareness, robustness, or personalization • Reflects impact of features like RAG, memory, testing, or safeguards (high-level)
Presentation, Communication, & Narrative	2	• Presentation is well-paced, smooth, and easy to follow • Clear, engaging, and confident delivery • strong verbal explanation & narrative • Audience can easily follow the demo and story
Reflection: Limitations & Future Improvements	1	• Acknowledges key limitations or gaps • Provides at least one realistic improvement or next step
Total	25	



Project Ideas

Ideas to help you brainstorm

Tech Focused Project Ideas

Project Idea	Description	Complexity
Knowledge Base Assistant (RAG App)	Chatbot that answers questions from uploaded PDFs (handbooks, case studies, manuals).	Multi-component RAG pipeline, ingestion, embeddings, indexing, UI, testing, guardrails.
Career Advisor & Learning Path Generator	Skill assessments, resume analysis, personalized career roadmaps, job-fit scoring.	Multi-step reasoning, data storage, integrations (job feeds), personalization logic.
Product Recommendation & Marketing Engine	Generates bundles, pricing suggestions, product descriptions, and marketing content for small business inventory.	Data cleaning, recommendation logic, content generation, dashboard building.
Compliance Checker / Policy Analyzer	Upload document → get flagged risks, compliance issues, rewritten safer versions.	Requires structured outputs, RAG accuracy tuning, safety constraints.
Meeting Analyst (Summaries + Actions + Risks)	Upload audio → summarize, extract tasks, risks, sentiment, stakeholder tone.	Multiple downstream workflows, multi-output tasks, transcription + analysis.
Customer Support Agent (Semi-Autonomous)	Auto-classifies customer queries, drafts responses, escalates, creates tickets in Jira/Zendesk.	Combines classification + generation + workflow automation + ticketing systems.
Lesson Planner / Curriculum Generator	Lesson plans, quizzes, assignments, curriculum maps; integrates with Google Classroom.	Multi-output content generation + structured templates + export workflows.
Legal Document Simplifier & Clause Reviewer	Detects risks, simplifies language, identifies missing clauses, provides rewrites.	Requires structured generation, multi-step analysis, document processing.

Consumer-Friendly Project Ideas

Project Idea	Description	Complexity
Personal Wellness & Habit Coach	AI assistant that generates customized habit plans, daily check-ins, wellness reflections, and progress tracking.	Multi-step reasoning, journaling memory, notification workflows, dashboards.
Smart Meal Planner & Grocery Assistant	Takes dietary preferences → generates recipes, weekly meal plans, grocery lists, and substitutes for allergies.	Multi-output generation + structured lists + integrations with recipe APIs.
Travel Planner & Itinerary Builder	Creates personalized travel itineraries, budgets, packing lists, and day-by-day schedules.	Retrieval from travel data, planning chains, multi-step tasks, export workflows.
Home Budgeting & Spending Insights Assistant	Users upload spending or connect via API → AI identifies patterns, categories, and gives savings strategies.	CSV ingestion, pattern recognition, dashboards, multi-output reasoning.
AI Home Décor Stylist	Users upload room photos → AI suggests décor styles, color palettes, furniture lists, and layout ideas.	Multimodal input, style classification, structured output prompts, RAG for inspiration.
AI Gift Recommendation Engine	User enters recipient details → AI generates personalized gift ideas, shopping lists, and budgets.	Preference modeling, product matching, integrations with e-commerce APIs.
Fitness & Workout Generator	Creates tailored workout plans based on goals, equipment, skill level, and weekly availability.	Rule-based + AI hybrid logic, multi-step plan generation, tracking dashboard.
AI Study Coach / Learning Companion	Helps students study: summaries, practice quizzes, spaced repetition reminders, flashcards.	Content generation, quiz creation, spaced-repetition workflow automation.

Consumer-Friendly Project Ideas

Project Idea	Description	Complexity
Parenting Support Assistant	Offers age-appropriate activity ideas, screen-time plans, parenting scripts, developmental milestone tracking.	Personalized content, structured recommendations, safety-guarded prompts.
Event Planner (Birthdays / Weddings / Parties)	Generates themes, budgets, vendor lists, timelines, invitations, and checklists.	Multi-step workflows, conditional planning, templated documents generation.
AI Home Maintenance Advisor	Gives seasonal maintenance checklists, appliance troubleshooting guides, and cost estimates.	RAG for home info, conditional logic, scheduling & reminders.
Pet Care Assistant	Provides training routines, feeding guides, symptom-based advice, enrichment activity ideas.	Multi-output generation, pet profile memory, classification + generation.
AI Cookbook Builder	Users upload handwritten or typed recipes → tool standardizes, formats, and builds a custom cookbook PDF.	OCR/cleanup pipeline, structured formatting, multi-recipe compilation.
Sleep Optimization Companion	Collects user sleep habits → recommends bedtime routines, improvements, and tracks patterns.	Data logging + AI insights + recommendation engine.
AI Fashion Stylist & Wardrobe Planner	Users upload wardrobe photos → AI suggests outfits, seasonal capsules, shopping gaps.	Multimodal inputs, tagging, outfit-generation logic, personalization.